

Reviewer Pack — Minimal Geometric Entropy of Tropical Curves and Communicati...

Entry dbc5a25e360b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 22:44:07 UTC

Minimal Geometric Entropy of Tropical Curves and Communication Complexity of Tensor Network Compression

Entry ID: dbc5a25e360b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 22:44:07 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Communication Complexity: Tensor Network Compression

Statement:

[‘For any instance of the Tensor Network Compression problem with $n \leq 40$ nodes, the communication complexity is upper-bounded by a function $f(n) = \alpha * \log(\beta^n)$, where α and β are constants.’, ‘The geometric entropy of the tropical curve corresponding to the tensor network compression instance is lower-bounded by a function $g(n) = \Omega(\log(\beta^n))$.’, ‘For all instances with $n \leq 40$, the ratio $f(n)/g(n)$ remains constant.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a framework for analyzing discrete structures in computational complexity. The geometric entropy of a tropical curve has been used to study communication complexity. Tensor network compression is a complex problem that may benefit from insights gained through tropical geometry.’, ‘This conjecture suggests that the complexity of tensor network compression can be related to the geometric structure of tropical curves, potentially leading to new algorithms or bounds in communication complexity.’]

Taxonomy category: TROPICAL_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3038c0a8c7974616

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For $n \leq 40$ nodes, we accept the conjecture if the ratio of communication complexity to geometric entropy ($f(n)/g(n)$) is within a margin of error of 0.01 for all generated instances.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "tensor network compression" - "geometric entropy" tropical curve" AND communication complexity" - "communication complexity" AND "minimal geometric entropy"

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1306.3497v2] The number of vertices of a tropical curve is bounded by its area - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def log_base(x, base):
        if x <= 0 or base <= 1:
            return None
        return math.log(x) / math.log(base)

    def communication_complexity(n):
        # Placeholder function for communication complexity
        return n * log_base(n, 2)

    def geometric_entropy(n):
        # Placeholder function for geometric entropy
        return log_base(n, 2)

    def ratio(n):
        alpha = 1.0
        beta = 2.0
        f_n = alpha * log_base(beta**n, 2)
        g_n = log_base(beta**n, 2)
        return f_n / g_n
```

```

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    cc = communication_complexity(n)
    ge = geometric_entropy(n)
    if cc is None or ge is None:
        continue
    results.append((n, cc, ge))

if not results:
    return {
        "metric_name": "Ratio",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No valid instances generated"
    }

total_ratio = sum(r[1] / r[2] for n, cc, ge in results)
mean_ratio = total_ratio / len(results)

return {
    "metric_name": "Ratio",
    "metric_value": mean_ratio,
    "instances_tested": len(results),
    "conjecture_holds": abs(mean_ratio - 1) < 0.01,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        result = f"SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}"
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = "Ratio does not remain constant"
        result = f"FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}"

    print(result)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4970ed74.py", line 78, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4970ed74.py", line 60, in run_trial
    total_ratio = sum(r[1] / r[2] for n, cc, ge in results)
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4970ed74.py", line 60, in <genexpr>
    total_ratio = sum(r[1] / r[2] for n, cc, ge in results)
                   ^
NameError: name 'r' is not defined. Did you mean: 're'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the ratio of communication complexity to geometric entropy meets the pre-registered support condition. | next: Review and debug the test code to ensure it runs successfully and produces the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13948
2	preregistration	ollama_remote	glm4:latest	0	0	9072
3	novelty	ollama_remote	glm4:latest	0	0	8737
4	novelty	ollama_remote	glm4:latest	0	0	9399
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15092
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6587
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14467
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7519
9	judge	ollama_remote	glm4:latest	0	0	15731

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 100554 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/dbc5a25e360b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dbc5a25e360b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/dbc5a25e360b.tar.gz` (if generated)